Análisis del Problema y Definición de Requerimientos del Sistema Enfocados en la Gestión de Memoria Estática y Dinámica en el Motor V8 de JavaScript y Node.js

Grupo 4

Freddy Alexander Ordoñez Gonzalez
Yimmy Onner Angulo Torres
Richar Santiago Guaman Sanchez
Erick Jahirs Roguel Baque
Joaquin Emilio Moscol Castillo

Carrera de Computación — Universidad Nacional de Loja — Tercer Ciclo

7 de octubre de 2026

Abstract

Resumen— La gestión eficiente de recursos es un aspecto crítico en el desarrollo de software moderno. En el ecosistema de JavaScript, el motor V8 —desarrollado por Google y empleado en plataformas como Google Chrome y Node.js— automatiza la ejecución y el almacenamiento mediante una arquitectura interna altamente optimizada. Este reporte de investigación examina el funcionamiento teórico del motor V8, enfocándose en la distinción operativa entre la **Pila (*Call Stack*)** y el **Montón (*Memory Heap*)**. Se detalla cómo la Pila procesa de forma secuencial y rápida los tipos de datos primitivos (*number, string, boolean, null, undefined, symbol*) bajo una estructura LIFO, mientras que el Montón maneja la asignación dinámica de objetos complejos, arreglos y clases. Asimismo, se aborda la intervención del Recolector de Basura en la prevención de fugas de memoria, proporcionando una base conceptual sólida para la optimización del rendimiento en aplicaciones informáticas.

Palabras clave: Motor V8, JavaScript, Node.js, Gestión de Memoria, Pila (*Call Stack*), Montón (*Memory Heap*), Recolector de Basura.

1 Introducción

Comprender cómo maneja la memoria un lenguaje de programación cambia por completo la forma en que se desarrolla software a nivel profesional. En el ecosistema de JavaScript, herramientas muy populares como Google Chrome y Node.js dependen directamente del motor V8. A diferencia de lo que ocurre en lenguajes como C o C++, donde el programador debe preocuparse manualmente por la asignación y liberación de los recursos, V8 se encarga de automatizar todo este proceso por debajo utilizando estructuras de datos muy eficientes.

A lo largo de este documento, el equipo de investigación se ha propuesto analizar a fondo la arquitectura interna de este motor. Específicamente, el estudio se centra en examinar las diferencias clave entre dos conceptos que resultan vitales:

- La **Pila (*Call Stack*)**, encargada de gestionar los tipos de datos primitivos y el orden de ejecución de las funciones.

- El **Montón (*Memory Heap*)**, que es el espacio donde realmente residen los objetos, arreglos y clases de mayor tamaño.

Bajo esta perspectiva grupal, analizar cómo interactúan estas dos áreas no solo ayuda a escribir código más limpio y optimizado, sino también a comprender mejor por qué ocurren problemas comunes como el consumo excesivo de recursos o las fugas de memoria, especialmente cuando las aplicaciones actuales manejan procesos concurrentes pesados mediante *Web Workers*.

2 Investigación Teórica

2.1 Introducción al Motor V8 de JavaScript

Para adentrarse en el análisis técnico, resulta fundamental comprender primero qué hay detrás del entorno de ejecución. A partir de la investigación realizada, se define que **V8 es un motor de código abierto de JavaScript desarrollado por Google** (escrito principalmente en C++) cuyo propósito inicial fue acelerar la ejecución de scripts dentro del navegador web Google Chrome. Sin embargo, su versatilidad ha permitido que hoy en día sea también el núcleo de entornos del lado del servidor tan potentes como Node.js.

La función principal de V8 es **compilar y ejecutar código JavaScript de manera sumamente eficiente**. Para lograrlo, implementa una técnica avanzada de compilación *Just-In-Time* (JIT), la cual traduce el código fuente directamente a lenguaje de máquina justo antes de ser ejecutado, evitando interpretaciones lentas.

Desde la perspectiva de la arquitectura interna que compete a este estudio, para administrar de forma correcta tanto la ejecución de las tareas como la memoria del sistema, V8 divide su espacio de trabajo en dos componentes estructurales principales:

1. **La Pila (*Call Stack*):** El espacio donde se controlan las llamadas a funciones y los datos de tamaño fijo.
2. **El Montón (*Memory Heap*):** La gran región de memoria dinámica destinada a albergar objetos complejos.

2.2 La Pila (*Call Stack*)

Definición y funcionamiento: La pila es una estructura de datos de tipo LIFO (*Last In, First Out* — Último en entrar, primero en salir). El motor de JavaScript utiliza la pila para realizar un seguimiento exacto de dónde se encuentra el programa en su ejecución, procesando una tarea a la vez (lo que se conoce como procesamiento monohilo o *single-threaded* en su núcleo).

¿Qué se almacena en la Pila?

- Los **tipos de datos primitivos** (como `number`, `string`, `boolean`, `null`, `undefined`, `symbol` y `bigint`). Estos elementos tienen un tamaño fijo conocido en tiempo de compilación y se almacenan directamente en la pila junto con su valor actual.
- Los contextos de ejecución y las llamadas a funciones (*function calls*), organizando de forma secuencial el orden en el que se van resolviendo las tareas del programa.

2.3 El Montón (*Memory Heap*)

Definición y funcionamiento: Es una gran región de memoria no estructurada (asignación dinámica) administrada por el recolector de basura (*Garbage Collector*) de V8.

¿Qué se almacena en el Montón?

- Los valores de referencia u **objetos complejos** cuyo tamaño no se conoce de antemano o puede cambiar dinámicamente, tales como objetos (`Object`), arreglos (`Array`), funciones y clases (`class`).
- En lugar de almacenar el valor completo, las variables ubicadas en la pila guardan únicamente una referencia o dirección de memoria que apunta al objeto real ubicado en el Montón.

2.4 Diferencias clave en la gestión de memoria

Para concluir esta sección teórica, es fundamental contrastar el comportamiento operativo entre ambas estructuras dentro de la arquitectura de V8, ya que cada una cumple un rol especializado:

- **Velocidad y Acceso:** La Pila ofrece un rendimiento de acceso sumamente rápido debido a su organización lineal y secuencial (LIFO), donde los datos se apilan y desapilan de forma inmediata. Por el contrario, el acceso al Montón es comparativamente más lento, ya que el motor debe seguir punteros y direcciones de memoria para localizar los objetos distribuidos de manera no estructurada.
- **Ciclo de vida y Liberación de Recursos:** En la Pila, la memoria se libera de forma automática y estricta tan pronto como una función termina su ejecución y abandona su ámbito. En cambio, el Montón maneja datos de ciclo de vida impredecible, por lo que depende totalmente de la intervención del Recolector de Basura (*Garbage Collector*) para identificar y barrer aquellos objetos huérfanos que ya no tienen referencias activas, evitando así saturaciones o fugas de memoria.
- **Naturaleza de los Datos:** Mientras que la Pila está diseñada exclusivamente para albergar valores primitivos de tamaño fijo y contextos de ejecución instantáneos, el Montón asume la responsabilidad de almacenar estructuras dinámicas, masivas y mutables que sustentan la lógica compleja de las aplicaciones web modernas.

A manera de recapitulación de lo analizado a lo largo de este punto, la información consultada permite comprender que el motor V8 de JavaScript no opera como un bloque aislado, sino como un sistema coordinado donde la Pila y el Montón se complementan perfectamente. Mientras la Pila mantiene el orden lógico y secuencial de la ejecución monohilo mediante datos primitivos, el Montón absorbe la complejidad dinámica de los objetos y referencias. Este entendimiento profundo resulta clave para dimensionar cómo los entornos modernos equilibran el rendimiento, la velocidad y la administración automática de los recursos en beneficio del software.

3 Diferencias Clave entre la Pila y el Montón

Para visualizar de manera sintética las discrepancias operativas y de almacenamiento entre ambos componentes del motor V8, se presenta a continuación la siguiente tabla comparativa:

Característica	**Pila (*Call Stack*)**	**Montón (*Memory Heap*)**
Tipo de datos	Primitivos (`number`, `string`, `boolean`, etc.)	Objetos, arreglos, funciones y clases.
Estructura	Ordenada y secuencial (LIFO).	No estructurada / libre (*dynamic allocation*).
Velocidad de acceso	Sumamente rápida.	Más lenta (requiere seguir referencias o punteros).
Gestión de memoria	Automática al salir del ámbito de ejecución.	Gestionada y liberada por el Recolector de Basura (*Garbage Collector*).

Tabla 1: Comparativa detallada entre las características operativas de la Pila y el Montón en V8.

3.1 Análisis Profundo de las Diferencias Arquitectónicas

Tras revisar la tabla anterior, como equipo pudimos notar cómo estas diferencias afectan directamente la forma en que escribimos y estructuramos nuestro código:

- **Cómo se guardan los datos:** En la Pila, las variables sencillas (como un número o un texto corto) guardan su valor directamente porque su tamaño se conoce de entrada. En cambio, en el Montón los datos son flexibles y pueden cambiar o crecer de tamaño (como una lista de elementos). Por eso, el Montón usa pequeñas direcciones que apuntan desde la Pila para ubicarlos fácilmente.

- **La velocidad al programar:** Notamos que la Pila es sumamente rápida porque solo va colocando o sacando elementos de manera ordenada. Por el contrario, el Montón requiere un poco más de esfuerzo del sistema para buscar los objetos, por lo que crear demasiados objetos sin control puede volver lenta una aplicación.
- **El manejo de las copias:** Al copiar un dato sencillo en la Pila, se crea un valor nuevo e independiente. Pero al trabajar con objetos en el Montón, lo que copiamos es solo una referencia o enlace; si modificamos el objeto en una variable, este cambio afectará a las demás variables que apunten al mismo sitio.

Como equipo pudimos comprobar que ambas estructuras se complementan: la Pila nos da la velocidad para ejecutar instrucciones paso a paso, y el Montón nos da la libertad de almacenar estructuras grandes y complejas.

4 Node.js y la División entre Datos Fijos y Dinámicos

Al llevar todo este análisis al entorno de servidor con **Node.js**, como grupo pudimos entender por qué la división entre datos fijos y dinámicos es tan importante para el diseño de sistemas. Cuando desarrollamos una aplicación backend, Node.js procesa solicitudes concurrentes, consultas a bases de datos y lógica de negocio en tiempo real utilizando el motor V8.

En este contexto, la distinción entre la Pila y el Montón se vuelve vital para el rendimiento del servidor: los datos fijos o primitivos que controlan el flujo inmediato de una petición viajan de forma rápida por la Pila, mientras que los grandes volúmenes de información, documentos JSON, arreglos de registros y conexiones dinámicas se alojan obligatoriamente en el Montón. Si como desarrolladores no diseñamos bien nuestros requerimientos y dejamos que el Montón crezca sin control con datos dinámicos innecesarios, Node.js sufrirá de bloqueos y caídas en el rendimiento debido al esfuerzo excesivo del recolector de basura. Por ello, comprender esta separación nos ayuda a plantear requerimientos de sistema mucho más estables y eficientes.

5 Limpieza de Basura y Optimización del Rendimiento

Al profundizar en la interacción entre el motor V8, la Pila y el Montón, como grupo identificamos que uno de los mayores retos en las aplicaciones actuales (ya sea en Google Chrome o en Node.js) es mantener un rendimiento fluido y evitar que la memoria se sature.

Aquí es donde cumple un papel estelar el **Recolector de Basura (*Garbage Collector*)**. Mientras la Pila se limpia sola de forma inmediata al terminar una función, el Montón acumula una gran cantidad de objetos temporales. El recolector de basura se encarga de revisar periódicamente qué objetos ya no se están utilizando en el programa para eliminarlos y liberar espacio. Si como desarrolladores dejamos referencias activas por error (creando lo que comúnmente llamamos fugas de memoria), el recolector no podrá limpiar esos datos, lo que provocará que la aplicación se vuelva lenta o termine consumiendo todos los recursos del equipo. Por lo tanto, comprender esta dinámica nos enseña la importancia de escribir código limpio, liberando variables y gestionando adecuadamente los recursos en nuestros proyectos.

6 Planteamiento del Problema del Proyecto ABP

6.1 Título y descripción del problema

Autocompletado Turístico de Loja con Política de Caché SIEVE. En la ciudad de Loja, la búsqueda rápida de sitios turísticos y comercios locales mediante sistemas de autocompletado web presenta latencias elevadas cuando se manejan consultas repetitivas de prefijos bajo alta concurrencia. Para solucionar este problema de rendimiento, el proyecto plantea desarrollar un sistema optimizado que combine estructuras de datos eficientes en memoria con una política de reemplazo de caché de vanguardia.

6.2 Objetivos

- **Objetivo General:** Diseñar e implementar un prototipo de autocompletado de sitios turísticos y comercios de Loja que utilice una estructura Trie y el algoritmo de caché SIEVE como política de reemplazo para optimizar búsquedas por prefijo.
- **Objetivos Específicos:**
  1. Construir en JavaScript un Trie acoplado a una memoria caché SIEVE basada en Mapa y Lista Doblemente Enlazada con evaporación por puntero *hand*.
  2. Diseñar una interfaz web en React con indicadores en tiempo real de latencia de consulta y tasa de aciertos (*Hit Ratio*) comparados frente a políticas tradicionales.

6.3 Estructuras, algoritmos y tecnologías

Elemento	**Papel en el sistema**
Trie (Árbol de Prefijos)	Almacena de manera óptima el vocabulario de sitios turísticos y comercios de Loja para permitir búsquedas instantáneas por prefijo.
Lista Doblemente Enlazada y Tabla Hash	Estructuras de soporte combinadas para gestionar los nodos del caché de manera enlazada y de acceso directo.
Algoritmo SIEVE	Política de reemplazo de caché eficiente (*USENIX NSDI 2024*) utilizada para desalojar elementos menos relevantes de forma rápida.
Node.js y Express	Entorno backend que ejecuta el motor V8, gestionando la concurrencia de peticiones y exponiendo los resultados mediante una API.
React y Tailwind CSS	Interfaz web interactiva para visualizar las sugerencias de autocompletado y las métricas de rendimiento en tiempo real.

Tabla 2: Estructuras, algoritmos y tecnologías seleccionadas para el desarrollo del proyecto ABP.

6.4 Datos estáticos y dinámicos

Tomando como base la investigación teórica sobre la gestión de memoria en el motor V8, el proyecto clasifica y administra sus elementos de información entre la Pila y el Montón de la siguiente manera:

Dato	**Tipo**	**Descripción**
Catálogo de sitios turísticos de Loja	Estático	Listado fijo inicial de ubicaciones, nombres y metadatos cargados al arrancar el servidor.
Estructura Trie de prefijos	Estático	Árbol de búsqueda principal construido en el arranque y alojado permanentemente en el Montón de V8.
Parámetros de configuración de SIEVE	Estático	Variables primitivas de control y tamaño máximo alojadas de forma rápida en la Pila.
Consultas de usuario en tiempo real	Dinámico	Cadenas de texto ingresadas por los usuarios que se procesan de manera secuencial en la Pila (*Call Stack*).
Nodos y resultados en la Caché SIEVE	Dinámico	Elementos de consulta reciente que se crean, mutan y eliminan dinámicamente en el Montón (*Memory Heap*).

Dato	Tipo	Descripción
Métricas de rendimiento (*Hit Ratio*)	Dinámico	Estadísticas de aciertos y latencia actualizadas de forma continua durante la ejecución del sistema.

Tabla 3: Clasificación de datos estáticos y dinámicos en relación con la memoria V8 del sistema.

6.5 Requerimientos funcionales

Nota: Las características descritas a continuación son ideas iniciales que podrán ir cambiando o mejorando a medida que vayamos avanzando con el desarrollo del proyecto.

Cód.	Prioridad	Descripción	Estructura
RF-01	Alta	Cargar la lista inicial con los sitios turísticos y comercios de Loja al encender el servidor.	Árbol Trie
RF-02	Alta	Buscar de inmediato el lugar turístico a partir de las primeras letras que escriba el usuario.	Árbol Trie
RF-03	Alta	Crear una memoria rápida (caché) con tamaño ajustable para probar diferentes formas de limpiar datos (SIEVE, LRU y FIFO).	Caché SIEVE / LRU / FIFO
RF-04	Alta	Revisar primero la memoria rápida al buscar; si no se encuentra, buscar en el árbol principal y guardarlo para futuras consultas.	Caché y Trie
RF-05	Media	Mostrar una barra de búsqueda en la pantalla que despliegue sugerencias mientras el usuario escribe sin saturar el sistema.	React Frontend
RF-06	Media	Permitir cambiar desde la interfaz la forma en que la memoria limpia los datos (política activa) en cualquier momento.	API REST Express

Tabla 4: Requerimientos funcionales preliminares del proyecto ABP.

6.6 Requerimientos no funcionales

Nota: Las características descritas a continuación son ideas iniciales que podrán ir cambiando o mejorando a medida que vayamos avanzando con el desarrollo del proyecto.

Cód.	Categoría	Descripción
RNF-01	Rendimiento	Las sugerencias de autocompletado deben mostrarse casi de inmediato, sin demoras perceptibles para el usuario.
RNF-02	Complejidad	Las búsquedas y actualizaciones dentro de la memoria rápida (caché) deben realizarse de forma directa y eficiente.
RNF-03	Memoria	El uso de la memoria RAM del sistema debe mantenerse controlado y dentro de un límite máximo establecido.

Cód.	Categoría	Descripción
RNF-04	Fiabilidad	Si el usuario escribe datos vacíos o extraños, el sistema debe responder limpiamente sin bloquearse ni mostrar errores críticos.
RNF-05	Mantenibilidad	El código debe estar organizado en partes independientes para facilitar las pruebas y comparaciones entre métodos.
RNF-06	Seguridad	El servidor debe limitar el tamaño máximo de lo que se escribe en la búsqueda para evitar saturaciones o bloqueos.

Tabla 5: Requerimientos no funcionales preliminares del proyecto ABP.

6.7 Alcance y supuestos

Incluye (preliminar): Modelado y carga del catálogo de sitios turísticos y comercios de Loja, gestión y búsqueda rápida de prefijos mediante la estructura Trie, implementación y comparación de las políticas de caché SIEVE, LRU y FIFO, exposición de una API backend con Node.js y visualización interactiva de métricas y sugerencias en una interfaz web con React.

No incluye (preliminar): Integración con bases de datos comerciales externas pesadas, pasarelas de pago o sistemas de geolocalización GPS en tiempo real. Los datos y las pruebas de alta concurrencia se simulan directamente en el entorno de desarrollo local.

7 Conclusiones

A lo largo de este reporte de investigación, el desarrollo de los temas abordados ha permitido al equipo consolidar aprendizajes fundamentales sobre el funcionamiento interno de JavaScript y su aplicación práctica en el diseño de sistemas. Haciendo énfasis en los pilares analizados y en el proyecto ABP, se concluye lo siguiente:

- **En cuanto al Motor V8 de JavaScript:** Se comprendió que detrás de plataformas populares como Google Chrome y Node.js existe una arquitectura muy sofisticada basada en compilación JIT, la cual transforma código interpretado en instrucciones de máquina altamente optimizadas. Esto resalta cómo los entornos modernos logran equilibrar velocidad y abstracción sin requerir una intervención manual de gestión de memoria por parte del programador.
- **En cuanto a la Pila (*Call Stack*):** Profundizar en su estructura secuencial de tipo LIFO permitió entender con claridad absoluta cómo el motor procesa instrucciones de manera monohilo (*single-threaded*) y cómo almacena los tipos de datos primitivos de tamaño fijo junto con los contextos de ejecución. Su inmediatez operativa resulta vital para mantener fluido el flujo del software.
- **En cuanto al Montón (*Memory Heap*) y su Rendimiento:** El análisis del Montón abrió paso a comprender la complejidad de manejar estructuras de datos dinámicas mediante sistemas de referencias, destacando la importancia crucial de la limpieza de basura para mantener la estabilidad y evitar saturaciones en el desarrollo de software.
- **En cuanto al Diseño del Autocompletado Turístico:** La integración del Árbol Trie con el algoritmo de caché SIEVE demostró ser una solución técnica altamente viable para resolver el problema de latencia en la búsqueda de sitios y comercios de Loja, optimizando de manera directa el uso de la memoria dinámica.
- **En cuanto a la Definición de Requerimientos y Node.js:** Plantear requerimientos funcionales y no funcionales claros bajo el entorno de Node.js garantiza que la arquitectura del sistema backend mantenga estabilidad, control de recursos y un rendimiento óptimo ante consultas concurrentes de los usuarios.

8 Referencias Bibliográficas

References

[1] H. Hong, S. Woo, S. Park, J. Lee, and H. Lee, "CIRCUIT: A JavaScript memory heap-based approach for precisely detecting cryptojacking websites," *IEEE Access*, vol. 10, pp. 95356–95368, 2022.

[2] C. R. Sampedro Guamán, S. A. Machuca Vivar, and E. I. Patrón Sabando, "Desarrollo de aplicaciones de escritorio híbridas con Javascript, CSS y HTML," *Mikarimin. Revista Científica Multidisciplinaria*, vol. 2, no. 2, pp. 85–94, May-Aug. 2016.

[3] M. Aigner, T. Hütter, H. Payer, A. Miller, C. M. Kirsch, and M. Preishuber, "ACDC-JS: Explorative benchmarking of JavaScript memory management," in *Proceedings of the 2014 ACM International Symposium on New Ideas, New Paradigms, and Reflections on Programming and Software (Onward!)*, Portland, OR, USA, 2014, pp. 67–78.

[4] Google, "Visualizing memory management in V8 engine (JavaScript, NodeJS, Deno, WebAssembly)," *Deepu.tech*, 2022. [En línea]. Disponible: https://deepu.tech/memory-management-in-v8

[5] Mozilla Developer Network (MDN), "Memory management — JavaScript," *MDN Web Docs*, 2022. [En línea]. Disponible: https://developer.mozilla.org/es/docs/Web/JavaScript/Guide/Memory$_m$*anagement*